CSET- 419 Introduction to Generative AI

SumMedi

Medical Text Summarization using Multi-Layer GraphRAG

Presented by:
Hari Nandhan Maddala (E23CSEU1134)
Richa Yanamandra (E23CSEU1249)

Problem Statement

Why traditional RAG fails in healthcare? → Flat vector search lacks structured reasoning

LLMs hallucinate due to:

- No ground-truth anchoring
- Limited context window

Medical data requires:

- High precision
- Explainability
- Evidence-backed response
-

Core Gap:
LLMs cannot reliably connect symptoms → diseases → treatments across documents.

Proposed Solution

Multi-Layer GraphRAG Architecture
We replace flat retrieval with a hierarchical knowledge graph

A Graph-based Retrieval-Augmented Generation system for safe and explainable medical AI

Built on:

- Neo4j Graph Database
- UMLS (3.5M concepts)
- PubMed (10K+ papers)
- Focus: Reducing hallucinations in medical LLMs

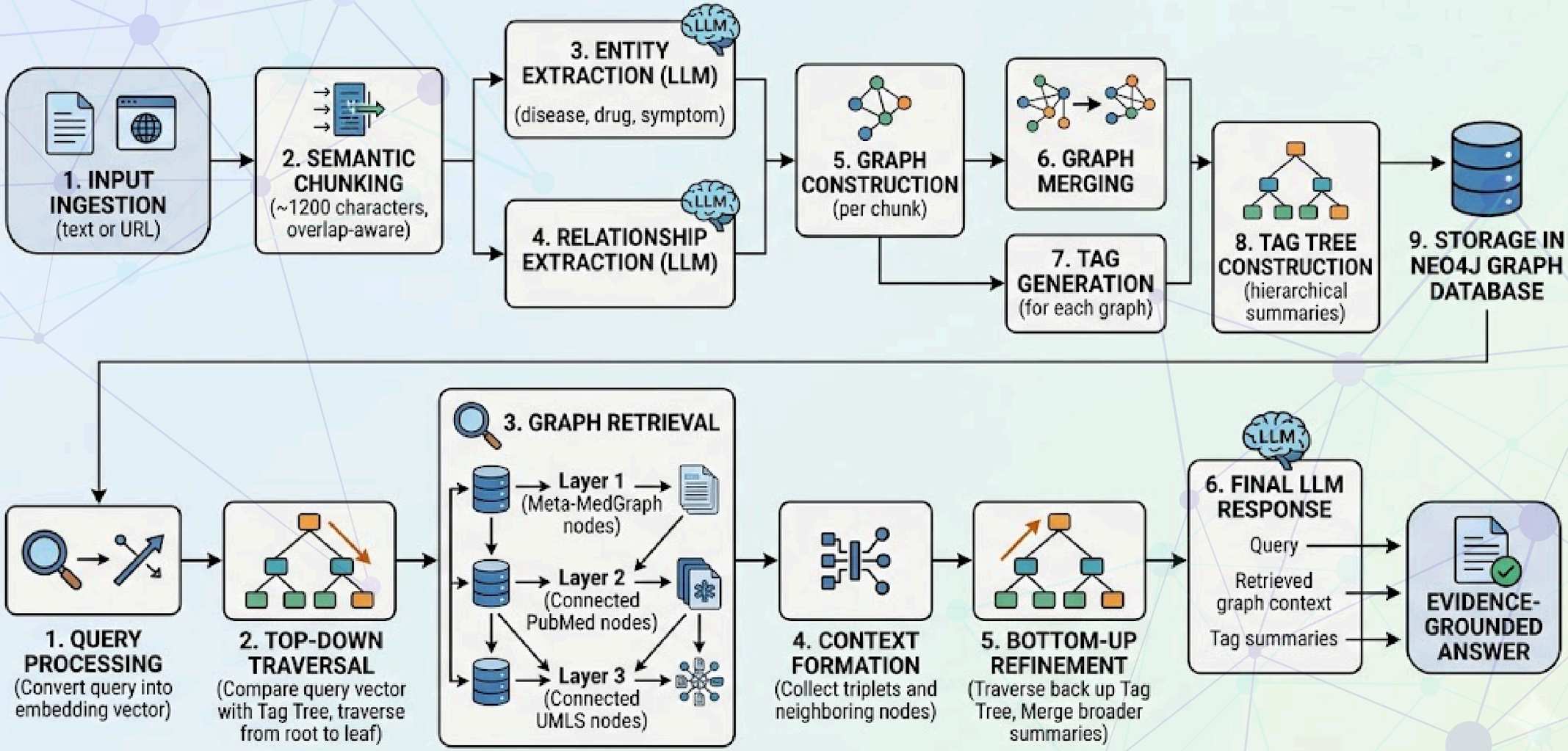
3 Graph Layers

Layer 1: User Data
Patient notes → dynamic graph (semantic triples)

Layer 2: PubMed
10,000+ research papers → clinical evidence

Layer 3: UMLS
3.5M medical concepts → ground truth ontology

SumMedi Project Workflow Pipeline



Methodology

From Text → Graph

1. **Semantic Chunking:** Overlap-aware chunks (~1200 characters)
2. **Entity Extraction (LLM):** Diseases, drugs, symptoms
3. **Relation Extraction**

Example: Metformin → treats → Diabetes

1. **Graph Construction:** Chunk-level graphs merged into global graph
2. **Tag Tree Generation:** Hierarchical summaries for fast retrieval
3. **Cross-Layer Linking:** Cosine similarity connects

Patient data ↔ PubMed ↔ UMLS

“U”-Retrieval Engine

Phase 1: Top-Down Traversal

- Query → vector embedding
- Navigate Tag Tree
- Reach most relevant leaf node

Phase 2: Triple-Neighbour Retrieval

Extract:

- Layer 1 (patient graph)
- Layer 2 (PubMed evidence)
- Layer 3 (UMLS definitions)

Phase 3: Bottom-Up Refinement

- Move back up Tag Tree
- Merge broader summaries
- Refine answer iteratively

Bottlenecks and Our Solutions

Scalability Bottleneck (3.5M nodes → 60GB RAM crash)

→ Implemented Database-Native Architecture (Neo4j) to offload graph storage and enable large-scale querying on low RAM

Slow Ingestion (8–10 min embedding delays)

→ Built Persistent Batch-Embedding Pipeline, reducing ingestion time to ~20 seconds

Inefficient Retrieval (shallow, context-limited answers)

→ Implemented U-Retrieval (Top-Down + Bottom-Up refinement) for precise and context-aware reasoning

Startup Latency (30+ sec index checks)

→ Added Asynchronous Background Indexing for instant app startup

Disconnected Data Sources (patient data, research, ontology not linked)

→ Created Triple-Layer Knowledge Graph enabling cross-layer reasoning in milliseconds

Results and Contributions

What we achieved:

- **Reduced hallucinations**: Answers grounded in UMLS + PubMed
- **Explainability:** Traceable graph paths for every answer
- **Scalability**: Millions of nodes via Neo4j indexing
- **Multi-hop reasoning**: Graph traversal > flat retrieval

- Multi-layer GraphRAG implementation
- Integration of UMLS + PubMed
- U-Retrieval mechanism
- Database native architecture

Thank You

The Project Repository can be found at:
https://github.com/richayanamandra/SumMedi